

Computer Vision: Machine Learning based Facial Recognition, Autonomous Driving, and Image Processing

TA: Shujian Yu

Date: 2026/03/29

- 今日目标

1. 了解常见的图像分割方法思路与优缺点
2. 学习并实践Otsu、分水岭、连通域、聚类分割及SLIC的使用
3. 掌握在Python/scikit-image或OpenCV中快速实现的思路

- 大纲

1. 图像分割简述
2. 阈值分割 (Otsu)
3. 连通域标记 & 形态学
4. 分水岭算法
5. 聚类分割 (K-means 与 Mean Shift)
6. 超像素 SLIC

1. 分割的目标

- 将图像划分成语义上有意义的若干区域或目标
- 为后续检测、分类、测量提供更精确的ROI (Region of Interest)

2. 常见挑战

- 对比度不足，前景/背景难以区分；光照不均、噪声和复杂纹理

3. 常见分割思路

- 阈值（全局、局部）
- 基于边缘或梯度
- 基于区域合并或区域生长（分水岭）
- 基于聚类（K-means、Mean Shift等）
- 超像素方法（SLIC等）



原图



实例级分割



语义级分割



全景分割

1. Otsu 核心假设：

- 图像灰度直方图呈双峰，分别对应前景和背景

2. 主要思想：

- 通过遍历可能阈值，找到能最大化类间方差 σ_b^2 的阈值
- 等价于最小化类内方差

3. 公式概览

$$\sigma_b^2 = \varpi_0(\mu_0 - \mu_T)^2 + \varpi_1(\mu_1 - \mu_T)^2$$

- ϖ_0 、 ϖ_1 = 前景/背景像素占比
- μ_0 、 μ_1 = 前景/背景灰度均值
- μ_T = 整幅图像灰度均值

Otsu 分割示例与局限

1. 流程示意

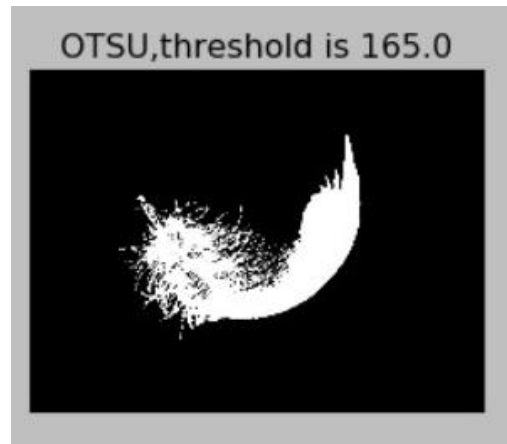
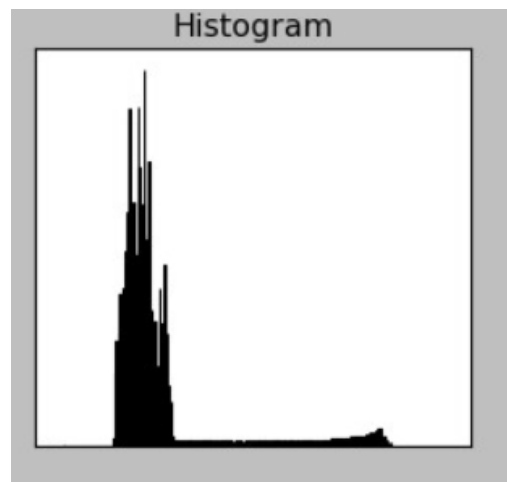
- 计算灰度图直方图
- 遍历每个可能阈值，计算类间方差
- 选出最大类间方差所对应的阈值，生成二值图

2. 对比图

- 上：原灰度图
- 中：灰度直方图带最佳阈值线
- 下：Otsu 分割结果

3. 局限

- 对噪声和不均匀光照敏感
- 多模态分布（多峰）场景效果不佳



1. 连通域标记

- 连通域一般是指图像中具有相同像素值且位置相邻的像素

点组成的图像区域

2. 形态学运算

- 开运算 (Opening): 去噪点
- 闭运算 (Closing): 填补小孔洞

3. 应用场景

- 后处理阶段合并小块或移除噪点
- 分割后可计算每个区域的面积、周长、圆度等

1) 前景二值图像



2) 连通区域分析方法标记后得到的label图像

Two-pass算法:



Seed-filling算法:



1. 概念类比

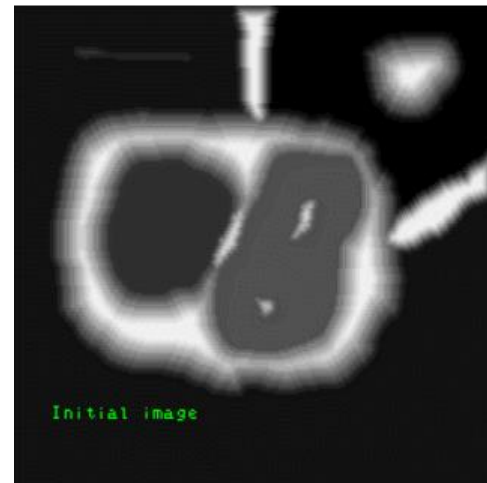
- 将图像看成地形，灰度值或梯度值 = “海拔”
- 水会从高处流向最低洼处，形成集水盆 (basin)

2. 基本步骤

- 在局部极小值处或指定标记点处开始
- 随着水位上升，浸没邻近像素
- 相邻“水域”之间的分水岭即为分割边界

3. 优点 & 缺点

- 优点：可得到完整、连贯的分割边界
- 缺点：易过分割；对噪声敏感



1. 分水岭常用流程

- 预处理（平滑或形态学）
- 距离变换 (Distance Transform)
- 选出局部最大值作为初始标记 (markers)
- 应用分水岭算法

2. 图示

- (a) 原图
- (b) 二值图或梯度图
- (c) 距离变换结果
- (d) 标记 + 分水岭分割后视觉化

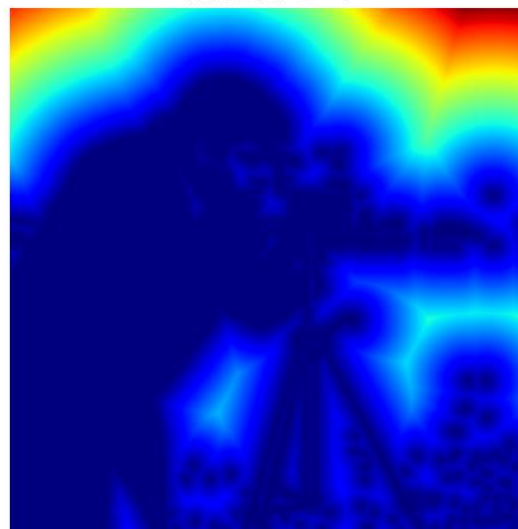
原图 (camera)



Otsu阈值后



距离变换 (EDT)



分水岭结果



1. K-means 聚类流程

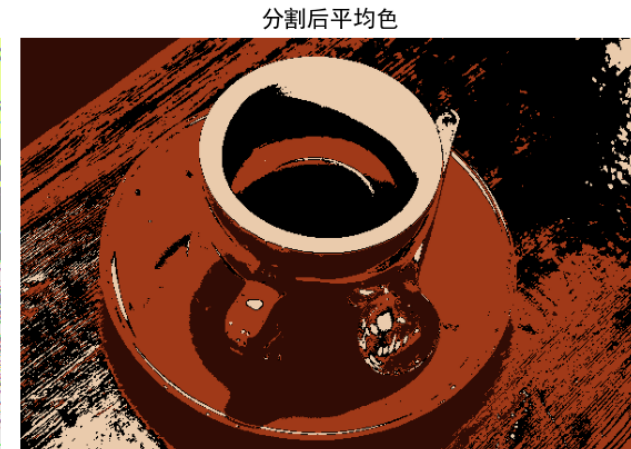
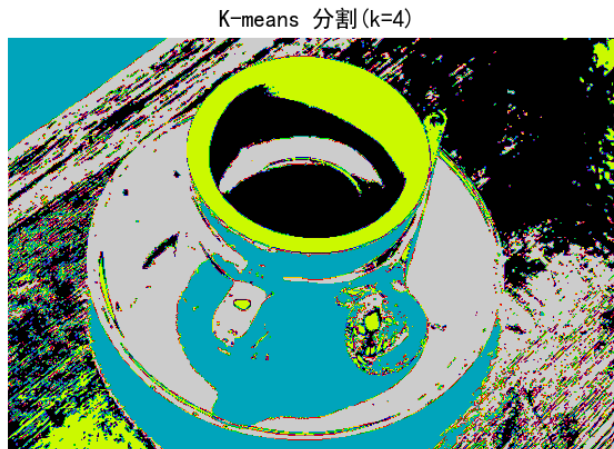
- 初始化 k 个聚类中心
- 分配像素到最近中心
- 更新中心 (均值)
- 迭代直到收敛

2. 在图像分割中的应用

- 将每个像素的 (R, G, B) 或 (L, a, b) 作为特征
- 聚成 k 个类，即得到 k 个分割区域

3. 优缺点

- 优点：实现简单、概念直观
- 缺点：需指定 k 、初始化易影响结果



1. Mean Shift 思想

- 非参数密度估计, 在特征空间中寻找密度峰

2. 过程简述

- 选定带宽(h)或核函数
- 对每个像素, 反复迭代计算Mean Shift向量并更新位置
- 收敛于某个峰值后, 像素被归为同一类

3. 对比 K-means

- Mean Shift 不需显式给出聚类数k
- 适合发现形状复杂的簇
- 计算量更大

原图 (flowers)



Quickshift分割结果



超像素 (Superpixels) 概念

1. 为什么要超像素

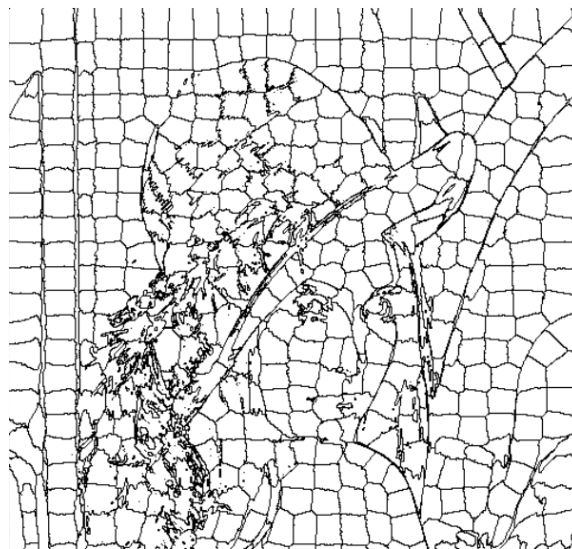
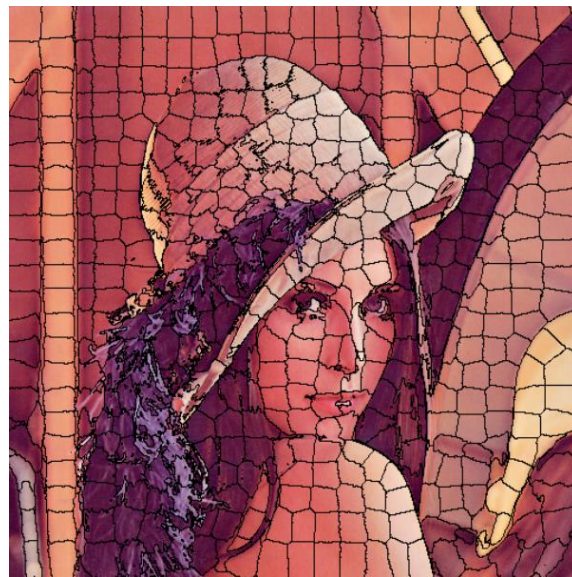
- 替代原始像素网格
- 减少计算量并保持较好边界贴合度

2. 主要要求

- 内部相对一致
- 与真实物体边界贴合较好
- 方便后续特征提取或更高级别处理

3. 常见方法

- SLIC (改进版K-means)
- 其他如: SEEDS、Felzenszwalb, Quickshift等



1. SLIC 核心思路

- 在 Lab+XY 空间做 k-means, 只在 25×25 的局部搜索
- 距离度量综合颜色距离和空间距离

2. 主要参数

- n_segments: 希望得到的超像素大致数量
- compactness: 平衡颜色和空间的权重

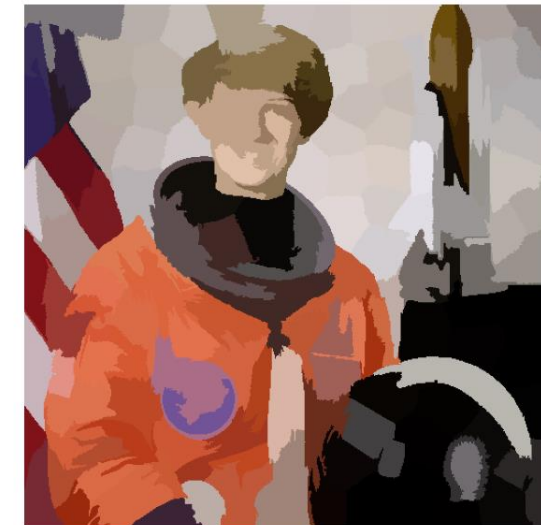
3. 优点

- 速度快, 内存开销小
- 易于实现, 广泛运用

原图 (flowers)



SLIC 超像素 (n_segments=200)



方法	优点	缺点	适用场景
Otsu	实现简单、快速	仅限双峰场景，对光照敏感	前景/背景分割较分明
分水岭	对边界精细	易过分割，需要选好marker	细胞、医学等精细场景
K-means	思路易懂、应用广	需给定k；对初始化敏感	基础多通道分割/聚类
Mean Shift	不用指定k；能发现复杂形状	算法慢，带宽选择敏感	中小图像，非参数聚类
SLIC 超像素	速度快，占用内存少，边界贴合度较好	参数需调；仍有过分割现象	作为预处理或中间层

要点回顾

- 没有“万能”方法，需结合具体图像特性
- 可多种方法组合（如 SLIC + K-means, Watershed + Morphology 等）

可讨论问题

- 什么时候全局阈值会失效？
- 如何在分水岭算法中减少过分割？
- SLIC 超像素与后续分割（如 Graph Cut / Deep Learning）怎样结合？

Computer Vision: Machine Learning based Facial Recognition, Autonomous Driving, and Image Processing

TA: Shujian Yu

Date: 2026/03/29